

Zero to Hero Study Handbook: computer-use

A from-scratch tutorial to this repository — what it is, the theory behind it, and exactly how the code implements that theory. Written for a reader who has never seen a Computer Use agent before and wants to end up able to read, run, and extend every layer of this codebase.

This repo currently ships **two API generations side by side**:

- **v1** — the original unversioned REST + WebSocket surface (`/api/agent/start`, `/ws`). Still fully implemented and running in `backend/server/__init__.py`, `backend/loop.py`, `backend/engine/*`.
- **v2** — a newer, typed, audited surface under `/api/v2` (`backend/v2/*`), with a five-tab TypeScript dashboard, deterministic route fallback, SQLite-backed session history, and a binary frame-streaming protocol.

v2 does not replace v1 in this snapshot — it *bridges into* the same v1 **AgentLoop** execution engine underneath (see Module 1, "How v1 and v2 relate"). Understanding v1 first makes v2 easy; this handbook is ordered that way.

Module 0: Concepts and Theory (start here if you're new to this)

What is a "Computer Use" agent?

A Computer Use (CU) agent is a large language model that has been given a special tool whose only job is to operate a computer the way a person would: look at a screenshot, decide on one action (click, type, scroll, press a key, wait), and receive a new screenshot showing the result. The model never sees or writes code to control the mouse — it emits a structured tool call like `{"action": "click", "x": 640, "y": 360}`, and a program outside the model executes that action for real and reports back.

This is different from:

- **Function calling for APIs** — there the tool wraps a business operation (e.g. `create_invoice(amount)`). A CU tool wraps *primitive physical input* (mouse/keyboard) against *visual output* (a screenshot).
- **Browser automation scripts** — those are pre-written, deterministic sequences (`click "#submit"`). A CU agent decides its next action itself, turn by turn, based on what it currently sees.

The active catalog contains three native Computer Use routes: Claude Sonnet 5 (`computer_20251124` through Anthropic Messages), GPT-5.6 Luna (the `computer` tool inside OpenAI Responses), and Gemini 3.6 Flash (Google Interactions Computer Use). Each has its own request/response shape, its own coordinate convention, and its own safety-confirmation mechanics. This repo's core architectural bet is: **absorb every one of those differences into small per-provider adapters, so the rest of the system — the loop, the executor, the sandbox, the UI — never has to know which vendor is running.**

The agentic loop (perceive → think → act)

Every CU run is the same three-step cycle, repeated until the model says it's done or a hard step limit is hit:

```
+-----+
| 1. PERCEIVE - capture a screenshot of the sandboxed |
|   desktop and hand it to the model as image input  |
|               |                                     |
|               v                                     |
| 2. THINK - the model reasons over the screenshot + task |
|   and emits exactly one structured tool call         |
+-----+
```

```

|           |           |
|           v           |
| 3. ACT - the executor translates that tool call into a |
|   real xdotool mouse/keyboard operation inside the   |
|   sandbox, waits for the UI to settle, and captures the |
|   next screenshot                                     |
|           |           |
|           +----> back to step 1                         |
+-----+

```

In this codebase that loop lives in `backend/loop.py::AgentLoop.run()` → `_run_computer_use_engine()`, which delegates the turn-by-turn mechanics to `backend/engine/__init__.py::ComputerUseEngine`.

Coordinate spaces (why Gemini math looks different)

Anthropic and OpenAI's tools speak in **real screen pixels** — a click at (640, 360) means the pixel 640 across, 360 down, on whatever resolution the sandbox is running. Gemini's tool speaks in a **normalized 0–999 space** regardless of actual resolution, so (500, 500) always means "the middle of the screen." `backend/engine/gemini.py` denormalizes those coordinates back to real pixels before they reach the executor; `DesktopExecutor` is built with a `normalize_coords` flag precisely so this one difference doesn't leak into the rest of the pipeline.

The sandbox — why every action goes through a second process

The model never controls your machine directly. Every action executes inside an isolated Ubuntu/XFCE Docker container (`docker/Dockerfile`, `docker/entrypoint.sh`) that exposes a tiny authenticated HTTP control plane, `docker/agent_service.py`, listening only on a container-internal port. The backend's `DesktopExecutor` (`backend/executor.py`) is an HTTP client to that service — it never runs `xdotool` itself. This means a misbehaving or adversarially-prompted model can, at worst, make a mess inside a disposable container, not your host.

Safety confirmation — the human-in-the-loop escape hatch

Some actions a model can propose (e.g. deleting files, submitting a payment form) are risky enough that this repo pauses and asks a human to approve before executing them. The handshake is a small piece of shared state in `backend/safety.py`: the loop "arms" a random nonce and an `asyncio.Event` for the session, broadcasts a `safety_confirmation` event over the WebSocket with that nonce, and blocks until either the caller echoes the same nonce back through the applicable v1 or v2 safety-decision endpoint (a constant-time comparison prevents another session from guessing it) or a timeout auto-denies the action. The v2 dashboard exposes approve/deny controls for `provider_default`, `confirm_mutating`, and `read_only` policies.

Provider-native web search (why it's not just another tool)

Adding a generic "search the web" tool alongside the computer tool would let the model split attention between two very different tool contracts every single turn, which measurably hurts CU tool-call reliability. So instead, when web search is enabled, this repo runs a **separate, one-shot planning call** before the CU loop even starts (`backend/providers/planner.py::create_web_execution_brief()`) using each vendor's own native search tool, and folds the resulting brief into the CU task text as plain instructions. The CU loop itself never sees a search tool — only the **computer** tool, every turn, with no competition.

v2 concepts: deterministic fallback, circuit breaking, and audited state

The v2 platform (`backend/v2/`) adds three ideas that v1 doesn't have:

- **Deterministic routing** — a v2 session names one primary "route" (provider + transport, e.g. `openai-direct`) and an explicit ordered list of fallback routes. There is no dynamic "pick the cheapest/fastest model" behavior; the operator decides the order.
- **Circuit breaking** — if a route fails repeatedly, it "opens" for a cooldown window so a session doesn't keep hammering a route that's currently down; `backend/v2/routing.py::CircuitBreaker`.
- **Audited, persistent history** — every v2 session, action, event, and latency metric is written to a local SQLite (WAL-mode) database (`backend/v2/persistence.py`) instead of living only in browser memory, so you can inspect what happened after the fact from the Audit tab.

The selected route is always one of `openai-direct` (GPT-5.6 Luna), `anthropic-direct` (Claude Sonnet 5), or `gemini-direct` (Gemini 3.6 Flash). All three support API-key credentials; Gemini can instead use a browser OAuth flow whose refreshable credential remains only in the process-local vault.

Glossary

Term	Meaning in this repo
CU	Computer Use — the agent pattern this whole project implements.
Turn	One perceive→think→act cycle of the agentic loop.
Action	One structured tool call (click, type, scroll, key, wait, ...).
Executor	The component that turns an action into a real xdotool operation (<code>DesktopExecutor</code>).
Coordinate space	Whether a provider's clicks are in real pixels (Claude/OpenAI) or normalized 0–999 (Gemini).
Route	(v2) A specific provider + transport + model combination a session can execute against.
Fallback	(v2) An ordered backup route tried if the primary route fails.
Circuit breaker	(v2) Temporarily stops retrying a route that has failed repeatedly.
Credential session	(v2) A short-lived ($\leq 8h$), process-memory-only holder of a provider API key or Google OAuth credential, never persisted to disk.
CUAF frame	(v2) The custom binary WebSocket frame format used to stream compressed screenshot previews (see Module 1).
Checkpoint	(v2) A provider-neutral snapshot of session progress (goal, confirmed step count, frame hash) — never raw model reasoning.
Safety confirmation	The nonce-gated human-approval handshake for risky actions.

Module 1: Foundations and Architecture

- **What this project does:** `computer-use` is a local full-stack workbench for provider-native Computer Use agents (Google Gemini, Anthropic Claude, OpenAI) running against a Dockerized Linux desktop.
- **Primary use cases:** browser automation, desktop app automation, screenshot-driven task execution, optional provider-native web-search planning, and optional document-grounded runs.
- **Runtime boundary:** the application targets one trusted operator on one workstation. FastAPI runs as a single process because the credential vault, active task handles, WebSocket clients, and circuit breaker state are all process-local, in-memory structures — there is no multi-worker or multi-tenant deployment mode.

Core paradigms and patterns used in this repo

- **Adapter pattern:** provider differences are normalized behind shared interfaces in `backend/providers/*.py`, `backend/engine/*.py`, and (for v2) `backend/v2/adapters.py`.
- **Orchestrator/loop pattern:** one session loop (`backend/loop.py::AgentLoop`) controls lifecycle, state transitions, callbacks, and termination for v1; a thin bridge (`backend/v2/orchestrator.py::V2Orchestrator`) tracks the same `AgentLoop` runs for v2 sessions (see "How v1 and v2 relate" below).

- **Event-driven architecture:** the backend broadcasts real-time events (`log`, `step`, `screenshot`, `agent_finished` for v1; JSON control events + binary CUA frames for v2) over WebSockets; the frontend reacts to them.
- **Protocol translation layer:** model tool actions are translated into desktop operations through `DesktopExecutor`, then sent over HTTP to `docker/agent_service.py`.
- **Schema-first validation:** Pydantic request/response models (`backend/models/schemas.py` for v1, `backend/v2/api.py`'s `ContractModel`-based classes for v2) plus runtime checks in handlers.
- **Deterministic fallback + circuit breaking (v2 only):** `backend/v2/routing.py::run_with_fallback()` tries routes in the order the caller specified, retries only what's explicitly marked retryable, and never dynamically re-ranks by cost or latency.
- **Defense in depth:** host allowlist, origin checks, optional shared workbench token (`CUA_API_TOKEN`), request size limits, action allowlists, and `_guard_api` middleware that gates API reads and writes when the token is configured.

How v1 and v2 relate

This is the one piece of architecture that's easy to misread from the directory layout, so it's worth stating plainly:

backend/v2/orchestrator.py does not contain a second execution engine. `V2Orchestrator.start()` just calls whatever "starter" function was registered with it. `backend/server/__init__.py` wires that starter to `_start_v2_execution()`, which constructs and runs a perfectly normal `backend.loop.AgentLoop` — the same class v1's `/api/agent/start` uses — and translates its result into a v2 `ExecutionOutcome`. So a v2 session still runs the v1 agentic loop, engine, executor, and sandbox underneath; v2 adds routing/fallback, a credential vault, SQLite audit history, and binary frame streaming *around* that same core, not a parallel one.

Architecture components and interactions

Layer	v1 and v2 relationship
HTTP endpoint	<code>backend/server/__init__.py</code> serves v1; <code>backend/v2/api.py</code> is mounted at <code>/api/v2</code> .
Session orchestration	v2 <code>V2Orchestrator</code> bridges into the same <code>backend/loop.py::AgentLoop</code> .
Provider and executor	<code>ComputerUseEngine</code> , provider adapters, <code>DesktopExecutor</code> , and the sandbox are shared.
Credentials	v1 resolves environment keys; v2 additionally supports process-local API-key and Google OAuth sessions.
Routing and persistence	v2 adds explicit ordered fallback, circuit breaking, SQLite WAL history, and retained audit frames.
Frame streaming	v1 uses <code>/ws</code> ; v2 uses coalesced binary CUA frames at <code>/api/v2/ws/{id}</code> .
Frontend	The shipped TypeScript dashboard uses only the v2 surface.

The **v1 frontend has been removed** from this snapshot (all of `frontend/src/pages/*`, `frontend/src/hooks/useSessionController.js`, `useWebSocket.js`, `frontend/src/api.js`, and friends were deleted as part of the JS→TypeScript rewrite). The v1 *backend* endpoints (`/api/agent/start`, `/ws`, etc.) are still fully implemented and tested — there simply isn't a shipped v1 UI anymore. The current frontend (`frontend/src/*.ts(x)`) only talks to `/api/v2/*`.

Main flow diagram (v1 execution core, shared by both surfaces)

```

v1 caller: any HTTP client → POST /api/agent/start, WS /ws
v2 caller: frontend/src/App.tsx → POST /api/v2/sessions, WS /api/v2/ws/{id}
|
v
FastAPI Server (backend/server/__init__.py)
|

```

```

+--> [v2 only] backend/v2/api.py router: validate contract, resolve
|      credential, call orchestrator.start() → _start_v2_execution()
|      |
|      v
+--> AgentLoop (backend/loop.py)                                <-- both surfaces
|      |                                                        converge here
|      v
|      ComputerUseEngine (backend/engine/__init__.py)
|      |
|      +--> Provider run wrapper (backend/providers/*.py)
|      |      |
|      |      +--> optional planning (backend/providers/planner.py)
|      |      +--> provider client loop (backend/engine/openai.py|claude.py|gemini.py)
|      |      |
|      |      v
|      |      DesktopExecutor (backend/executor.py)
|      |      |
|      |      v
+--> In-container Agent Service (docker/agent_service.py) via /action and /screenshot
|      |
|      v
      Xvfb + XFCE desktop + browsers/apps (docker/entrypoint.sh, docker/Dockerfile)

```

Realtime back-channel:

```

v1: AgentLoop -> FastAPI WS broadcast (/ws) -> (no shipped v1 UI in this snapshot)
v2: FrameBroker (backend/v2/frames.py) -> /api/v2/ws/{id} -> useLiveStream.ts -> App.tsx

```

Module 2: Repository Map

- **Application boundary:** startup, public-bind guardrail, REST/WS/noVNC authentication, and execution bridge in `backend/main.py` and `backend/server/__init__.py`.
- **Contracts and catalog:** Pydantic request contracts and the three-route model catalog in `backend/models/`, `backend/v2/models.py`, and `backend/v2/api.py`.
- **Shared execution:** provider-native turns, optional planning, action normalization, and sandbox calls in `backend/loop.py`, `backend/engine/`, `backend/providers/`, and `backend/executor.py`.
- **Safety and files:** nonce confirmations and provider-aware attachment handling in `backend/safety.py`, `backend/files.py`, and `backend/infra/storage.py`.
- **v2 services:** ordered fallback, API-key/OAuth vault, SQLite audit, frame retention, and CUAf streaming in `backend/v2/routing.py`, `credentials.py`, `persistence.py`, `retention.py`, and `frames.py`.
- **Sandbox:** isolated desktop, allowlisted input, and token-protected action service in `docker/agent_service.py`, `docker/entrypoint.sh`, and `docker-compose.yml`.
- **Frontend:** five-tab v2 workbench, token-aware API/WS clients, and preview decoding in `frontend/src/App.tsx`, `api.ts`, `useLiveStream.ts`, and `protocol.ts`.

- **Tooling:** local configuration, development orchestration, and release/documentation builds in `.env.example`, `dev.py`, `setup.*`, and `scripts/`.
-

Module 3: Core Execution Flows

Flow A (v1): Start agent — REST call to running session

1. Any HTTP client sends `POST /api/agent/start` with `task`, `provider`, `model`, and optional `max_steps/reasoning_effort/use_builtin_search/attached_files`.
2. `backend/server/__init__.py::api_start_agent()` validates:
 - `engine == "computer_use"`, `execution_target == "docker"`
 - `provider/model` against the v1 allowlist
 - `max_steps` hard cap (200)
 - attachments via `backend/files.py::validate_attached_files()`
 - reasoning/search config via `validate_builtin_search_config()`
 - API key via `resolve_api_key()` (UI input → `.env` → system env)
3. Backend starts/checks the sandbox via `backend/infra/docker.py::start_container()`. A 409 here means the container process exists but the in-container agent service isn't ready yet — the backend re-checks readiness rather than trusting a stale cached state.
4. Backend creates `AgentLoop(...)`, stores it in `_active_loops`, and schedules `_run_and_notify()` as a background asyncio task.
5. API returns the session handle immediately; the run continues async.

Start request/response shapes:

```
// POST /api/agent/start
{
  "task": "string", "api_key": "string|null", "model": "string",
  "max_steps": 50, "engine": "computer_use",
  "provider": "google|anthropic|openai", "execution_target": "docker",
  "reasoning_effort": "none|minimal|low|medium|high|xhigh",
  "use_builtin_search": true, "attached_files": ["f_..."]
}

// 200 response
{"session_id": "uuid", "status": "running", "engine": "computer_use", "provider": "google|anthropic|openai"}
```

Flow B (v1): Session loop and provider execution

1. `AgentLoop.run()` sets status to `RUNNING`.
2. `AgentLoop._run_computer_use_engine()` maps the provider string to the `Provider` enum and builds the system prompt via `backend/prompts.py::get_system_prompt()`.
3. `ComputerUseEngine.execute_task()` delegates to `backend/providers/run_client(...)`.
4. The provider wrapper (`backend/providers/openai.py`, `anthropic.py`, `gemini.py`) can run `maybe_plan_with_web_search(...)` first if search is enabled (see Module 0 — this is a separate call, not a competing tool).
5. The provider client emits turn records/logs; `AgentLoop._on_turn()` maps action data to a `StepRecord`, running a stuck-agent fingerprint check (three identical consecutive actions triggers a clean stop).
6. Backend broadcasts a `step` event to `/ws` clients after each turn.
7. Session ends as `COMPLETED`, `STOPPED`, or `ERROR`; backend broadcasts `agent_finished`.

Flow C (v1 and v2 — shared): Tool action → real desktop operation

1. A provider action reaches `DesktopExecutor.execute(name, args)` (`backend/executor.py`).
2. `execute()` dispatches to a handler by name convention: `getattr(self, f"_act_{name}", None)` — e.g. `_act_click_at`, `_act_type_text_at`.
3. The handler POSTs to `/action` on the in-container agent service.
4. `docker/agent_service.py:AgentHandler.do_POST()` validates the shared `AGENT_SERVICE_TOKEN`, resolves any action-name alias via `backend/models/registry.py`: checks `_is_action_enabled()` against the allowlist, and dispatches to the real `xdotool/scrot` call.
5. The result is cached briefly by `action_id` so a retried/duplicated request replays the same result instead of re-executing (idempotency).
6. The executor wraps the result as a `CUActionResult` and returns it upstream.

```
// POST /action (inside the sandbox)
{
  "action": "type_text_at", "coordinates": [640, 360], "text": "hello world",
  "press_enter": true, "clear_before": true, "mode": "desktop",
  "action_id": "uuid:0", "include_screenshot": 1
}
```

Flow D (v1): WebSocket event stream (/ws)

1. A client opens `/ws`; the backend enforces the same origin/token gate (`_ws_origin_ok`, `_ws_token_ok`) used by the noVNC proxy.
2. The client can send `{"type": "screenshot_mode", "mode": "on"|"off", "session_id": "...}"` to subscribe/unsubscribe from the shared screenshot publisher loop (one capture loop fans out to every subscriber — not one loop per client).
3. Backend emits events validated against `backend/server/ws_schema.py`: `screenshot`, `screenshot_stream`, `log`, `step`, `agent_finished`, `pong`.

(Note: the v1 UI that consumed this stream has been removed from this snapshot — this endpoint remains fully implemented and tested, but the current frontend only speaks v2. See Flow H below for the live surface.)

Flow E (v1): File upload and provider-specific attachment handling

1. A client POSTs a file to `/api/files/upload`.
2. Backend streams it into `FileStore.add_stream()` on disk, keyed by an opaque `file_id`.
3. `file_ids` are passed back in `attached_files` on `/api/agent/start`.
4. `validate_attached_files(provider, file_ids)` enforces format/existence/dedup and provider compatibility.
5. Provider paths differ deliberately:
 - **OpenAI**: `prepare_openai_file_search()` creates a vector store and uploads files, attaching `file_search`.
 - **Anthropic**: `prepare_anthropic_documents()` uploads `.pdf/.txt` via the Files API; `.md/.docx` are extracted and inlined as text.
 - **Gemini**: rejected outright for CU runs (`GEMINI_CU_FILE_REJECTION`) — Gemini's File Search tool cannot be combined with the Computer Use tool.

Flow F (v1 and v2 — shared): Safety confirmation handshake

1. The provider requests confirmation for a risky action; the loop calls `safety_registry.arm(session_id)`, which mints a fresh nonce and a cleared `asyncio.Event`.
2. Backend emits a `safety_confirmation`-shaped event with `session_id`, `nonce`, and `explanation`.
3. The caller answers via `POST /api/agent/safety-confirm` for v1 or `POST /api/v2/sessions/{id}/safety-decisions` for v2 with the same `nonce`.
4. Backend verifies the nonce with `hmac.compare_digest` (constant-time, so an unrelated caller can't brute-force or timing-attack another session's prompt), records the decision, and unblocks the loop.

5. A timeout path auto-denies if nobody answers in time.

Flow G (v2 only): Create a session with routing and a credential vault

1. The dashboard's Providers tab either calls `POST /api/v2/credential-sessions` with a raw API key or starts Google OAuth at `/api/v2/credential-sessions/google`. The vault stores the resulting credential only in process memory (never on disk or in the audit log) and returns an opaque credential-session id with a TTL capped at 8 hours.
2. The Live tab calls `POST /api/v2/sessions` with `task`, `model`, an explicit `primaryRoute`, an ordered `fallbackRoutes` list, and that `credentialSessionId`.
3. `create_session()` (`backend/v2/api.py`) resolves the model from `ModelCatalog`, validates the primary + fallback routes are compatible, writes a `sessions` row via `SqliteStore.create_session()`, and kicks off an async `_coordinate()` task.
4. `_coordinate()` calls `run_with_fallback()` (`backend/v2/routing.py`) over the ordered route list. For each route, `_invoke()` resolves a credential — the vault first, falling back to the environment variable for that provider if no vault session was supplied — then calls `V2Orchestrator.start()`, which runs the same `AgentLoop` v1 uses.
5. On success, confirmed actions are journalled one-by-one via `SqliteStore.append_action()`, a `ROUTE_SUCCEEDED` event is recorded, and a provider-neutral checkpoint is saved (goal + confirmed step count only — never raw vendor reasoning). On failure, a `RouteFailure` marks the route's circuit breaker and — if the failure is retryable — the next fallback route in the list is tried.

Flow H (v2 only): Live binary frame streaming

1. The frontend opens `wss://.../api/v2/ws/{sessionId}` from `useLiveStream.ts`, optionally appending `?token=` if the session-scoped workbench token is configured from `CUA_API_TOKEN`.
2. `v2_websocket_endpoint()` sends a `SESSION_STREAM_READY` JSON event, then loops: capture a frame via the shared `FrameBroker` (`backend/v2/frames.py` — concurrent callers share one in-flight capture instead of triggering redundant screenshots), send a `FRAME` JSON metadata event, then send the actual pixels as a binary CUAF frame (`pack_cuaf_frame()` — a fixed 30-byte header: magic, version, codec, sequence, width, height, timestamp, followed by the raw WebP/JPEG bytes).
3. `frontend/src/protocol.ts::decodeCuafFrame()` parses that header back out and hands the frontend an object URL to paint into `<img>`.
4. If audit-frame retention is enabled for the session, the canonical (uncompressed) frame is also written to disk via `FrameRetentionStore.put()`, hashed and content-addressed, subject to the 7-day / 1 GiB eviction policy.

Module 4: Setup and Run Guide

Prerequisites

Requirement	Version	Why
Docker Desktop / Engine	24+	Runs the <code>cua-environment</code> sandbox
Python	3.12–3.14	Backend typing features, asyncio task groups
Node.js	22+	Vite 6 dev server
Provider sign-in	OpenAI/Anthropic/Google API key, or Google OAuth	At least one required

Environment setup

On Windows 11, double-click `START.bat`. It installs missing Docker Desktop, Node.js LTS, and uv through winget; creates `.env` if needed; generates the required sandbox secrets; installs locked dependencies; builds the sandbox; and opens the dashboard. Existing `.env` values are preserved. If Docker asks for a restart or

initial WSL setup, complete it and run `START.bat` again.

For manual or non-Windows setup:

1. Copy `.env.example` to `.env`.
2. Set required sandbox secrets: `AGENT_SERVICE_TOKEN=...`, `VNC_PASSWORD=...` (generate unique random values — don't reuse examples).
3. Sign in from the Provider Manager with an OpenAI, Anthropic, or Google API key; alternatively configure Google OAuth.
4. Optional: `HOST/PORT`, `CUA_V2_DB_PATH` (v2 SQLite location, defaults to `data/computer-use-v2.sqlite3`), `CUA_V2_FRAME_PATH`, `CUA_API_TOKEN` + `CUA_ALLOW_PUBLIC_BIND=1` for external binding, `CORS_ORIGINS`, `CUA_ALLOWED_HOSTS`.

Typical command sequences

Windows one-click install and launch:

```
.\START.bat
```

Manual bootstrap and launch:

```
bash setup.sh           # or setup.bat on Windows
```

Path B — manual daily startup:

```
uv sync --frozen
Set-Location frontend; npm ci; Set-Location ..
.\dev.bat           # or: uv run --frozen python dev.py
```

Runtime access

- UI: `http://localhost:3000` (dev) or `http://127.0.0.1:8100` (built, single-process — FastAPI serves `frontend/dist` once it exists)
- Backend default port: 8100
- OpenAPI docs: `/docs`
- noVNC interactive desktop: `/vnc/vnc.html?...`

Database / external state

- v1 keeps everything in memory — nothing survives a restart.
- v2 persists to a local SQLite WAL database at `CUA_V2_DB_PATH` (default `data/computer-use-v2.sqlite3`); back it up alongside its `-wal/-shm` files. Credential-vault entries are intentionally *not* recoverable after a restart — that's the point. Retained audit frames live under `CUA_V2_FRAME_PATH` (default `data/audit-frames`) and are bounded to seven days or one GiB.
- The only external runtime dependency is the Dockerized desktop sandbox plus whichever upstream provider APIs you configure.

Regenerating the companion HTML/PDF

```
pandoc -f gfm -t html5 docs/zero-to-hero-study-handbook.md -o docs/zero-to-hero-study-handbook.html
pandoc -f gfm -t pdf   docs/zero-to-hero-study-handbook.md -o docs/zero-to-hero-study-handbook.pdf
```

Module 5: Study Plan and Practice Exercises

Ordered study plan

1. Read Module 0 above until you can explain the perceive→think→act loop and why coordinate spaces differ per provider, from memory.
2. Read `backend/models/schemas.py` (v1) and `backend/v2/api.py`'s `ContractModel` subclasses (v2) to lock in the data contracts.
3. Read `backend/server/__init__.py`, especially `api_start_agent()`, `websocket_endpoint()`, `v2_websocket_endpoint()`, and `_guard_api()`.
4. Read `backend/loop.py` for session lifecycle, then `backend/v2/orchestrator.py` to see how thin the v2 bridge actually is.
5. Read `backend/engine/__init__.py` and one provider file (`backend/engine/openai.py`).
6. Read `backend/executor.py` and `docker/agent_service.py` together.
7. Read `backend/v2/routing.py` (fallback + circuit breaker) and `backend/v2/credentials.py` (vault) together.
8. Read `backend/v2/frames.py` and `frontend/src/protocol.ts` together — they're two halves of the same binary format.
9. Read `frontend/src/api.ts`, `frontend/src/useLiveStream.ts`, and `frontend/src/App.tsx`.
10. Read infra files: `backend/infra/config.py`, `backend/infra/docker.py`, `.env.example`, `docker-compose.yml`.
11. Review tests: `tests/test_server.py`, `tests/test_v2_platform.py`, `tests/test_provider_run_contract.py`, `tests/docker/test_agent_service.py`.

Practice exercises with model answer outlines

1. **Trace how `use_builtin_search` changes behavior end-to-end.** Files: `backend/server/__init__.py`, `backend/providers/_common.py`, `backend/providers/planner.py`. Outline: request flag validated → provider wrapper runs a one-shot planning call with the vendor's native search tool → planner brief is appended into the CU task text; the CU loop itself never sees a search tool.
2. **Explain why `POST /api/agent/start` can return 409 even when the container process exists.** Files: `backend/server/__init__.py`, `backend/infra/docker.py`. Outline: the container process can exist while the in-container agent service is still `unready`; the backend re-checks readiness state before creating a session rather than trusting a stale cache.
3. **Map one `click_at` action from model output to final execution.** Files: `backend/loop.py`, `backend/executor.py`, `docker/agent_service.py`. Outline: turn action mapped to an executor handler by name convention → executor POSTs `/action` → agent service resolves any alias, checks the allowlist, dispatches to the real `xdotool` call.
4. **Explain the safety nonce handshake and anti-replay guard.** Files: `backend/loop.py`, `backend/safety.py`, `backend/server/__init__.py`. Outline: backend arms a nonce + event; caller echoes the nonce; backend verifies it with a constant-time comparison before setting the decision, so a different session can't resolve this one's prompt.
5. **Explain how a v2 session picks between its primary and fallback routes, and what makes a failure "retryable."** Files: `backend/v2/api.py::create_session`, `backend/v2/routing.py::run_with_fallback()`. Outline: routes are tried strictly in caller-specified order; `RouteFailure.retryable` and the circuit breaker's open/closed state jointly decide whether the same route is retried or the loop moves to the next fallback.
6. **Explain why a v2 credential session is never persisted, and what happens if you resolve one after it's expired.** Files: `backend/v2/credentials.py::Credential`. Outline: secrets live only in a process-memory dict wrapped in Pydantic `SecretStr`; `resolve()` returns `None` past the TTL and the entry is dropped, so an expired session can never leak a stale key.
7. **Decode one CUAF binary frame by hand.** Files: `backend/v2/frames.py::pack_cuaf_frame()`, `frontend/src/protocol.ts::decodeCuafFrame()`. Outline: 4-byte magic "CUAF", 1-byte version, 1-byte codec, 8-byte sequence, 4-byte width, 4-byte height, 8-byte timestamp, then raw image bytes — walk both implementations and confirm the offsets agree.
8. **Compare OpenAI vs. Anthropic file-attachment ingestion.** Files: `backend/files.py`, `backend/engine/openai.py`, `backend/engine/claude.py`. Outline: OpenAI uses a vector store + `file_search`; Anthropic uses the Files API for PDF/TXT and inlines extracted text for MD/DOCX; Gemini rejects

attachments for CU runs entirely.

9. **List three backend hardening controls that apply to both v1 and v2 mutating routes.** Files: `backend/main.py`, `backend/server/__init__.py`.
Outline: external-bind guardrail in `main.py`; host-allowlist middleware; the same origin/token gate (`_require_origin`, `_require_rest_auth`) applied to every v1 POST handler individually and to API reads and writes through `_guard_api` when the workbench token is configured.
-

Understanding Verification Checklist

- Can you explain the perceive→think→act loop and name the file/function where it actually iterates?
- Can you explain why Gemini's coordinates need denormalizing but Claude's and OpenAI's don't?
- Can you explain, in one sentence, what `V2Orchestrator` actually adds on top of `AgentLoop` — and what it deliberately does *not* reimplement?
- Can you map a provider tool action all the way to `_dispatch_desktop()` in `docker/agent_service.py`?
- Can you explain the safety nonce handshake and its timeout behavior end-to-end, for both v1 and v2 sessions?
- Can you explain why a v2 credential session has an 8-hour cap and is never written to disk?
- Can you explain how `run_with_fallback()` decides whether to retry the same route or advance to the next fallback?
- Can you decode the fixed-size header of a CUAF binary frame from memory?
- Can you list where and why 400, 401, 403, 409, 422, 429, and 503 appear across the v1 and v2 surfaces?
- Can you narrate the full architecture — from `backend/main.py`, through whichever API version handles a request, down through `AgentLoop` and the sandbox, and back up to whichever frontend surface renders it — without skipping a layer?